

Drunken Merge Sort

A randomized, block-fusion sorting algorithm (novelty / educational)

Status: Educational / portfolio exploration piece — not intended as a production-grade sorting algorithm. This document specifies the algorithm precisely, proves it terminates and is correct, and analyzes its (bad) expected-time complexity honestly.

1. Intuition

Drunken Merge Sort starts by treating every element as its own tiny "block." On each round, it randomly shuffles the current sequence of blocks — like a drunk person stumbling into a new arrangement. After each shuffle, it looks for any two **adjacent** blocks that already happen to be in sorted order relative to each other, and permanently fuses them into one bigger block ("sobering up"). It also lets a lone element get absorbed into a neighboring block if the element's value fits inside that block's range. This repeats until only one block is left — which, by construction, must be the fully sorted array.

2. Definitions

- **Block:** a contiguous logical unit $B = (\text{elements}, \text{min}, \text{max})$ where `elements` is an internally-sorted list, $\text{min} = \text{elements}[0]$, and $\text{max} = \text{elements}[-1]$.
- **Fusible pair:** two blocks A, B that are *adjacent in the current sequence* (A immediately followed by B) and satisfy $\text{max}(A) \leq \text{min}(B)$.
- **Absorbable element:** a single-element block x adjacent to a block B such that $\text{min}(B) \leq x \leq \text{max}(B)$ (x is inserted at its correct sorted position inside B).
- **Stumble:** one random permutation of the current block sequence.

- **Sober block:** a block that has been fused at least once. Fused blocks never break apart again.
-

3. Pseudocode

```

function DrunkenMergeSort(array A of n distinct elements, max_stumbles):
    blocks ← [ Block([x]) for x in A ]      # every element starts
alone
    stumble_count ← 0
    event_log ← []

    while length(blocks) > 1:
        if stumble_count >= max_stumbles:
            return FAILURE("passed out before finishing")

        blocks ← random_permutation(blocks)  # the "stumble"
        stumble_count ← stumble_count + 1
        event_log.append(("stumble", stumble_count, snapshot(blocks)))

        changed ← true
        while changed:
            changed ← false

            # Pass 1: fuse adjacent sorted-compatible blocks
            for i from 0 to length(blocks) - 2:
                A ← blocks[i]
                B ← blocks[i+1]
                if A.max <= B.min:
                    fused ← Block(A.elements + B.elements) # already
sorted
                    replace blocks[i..i+1] with [fused]
                    event_log.append(("fuse", A, B, fused))
                    changed ← true
                    break # restart scan; indices shifted

            # Pass 2: absorb a lone element into an adjacent block's
range
            for i from 0 to length(blocks) - 1:
                if blocks[i] is single-element and i has a neighbor B:
                    x ← blocks[i].elements[0]
                    if B.min <= x <= B.max:
                        insert x into B.elements at correct sorted

```

```

position
        remove blocks[i]
        event_log.append(("absorb", x, B))
        changed ← true
        break

return (blocks[0].elements, stumble_count, event_log)

```

Notes on the pseudocode:

- Fusion only ever concatenates two already-internally-sorted lists whose ranges don't overlap and are in order — so the result is trivially sorted without needing a merge step.
- The inner `while changed` loop deterministically collapses everything fusible *within one stumble* before the next random shuffle — this is what keeps the algorithm from re-shuffling progress away needlessly.
- `max_stumbles` is a required safety cap. Without it, worst-case runs do not reliably terminate in any practical time.

4. Correctness

Claim: If DrunkenMergeSort terminates with one block, that block is `A` sorted.

Proof sketch (invariant): Every block, at all times, has an internally sorted `elements` list (true initially — singletons are trivially sorted; preserved by fusion, which only concatenates two sorted lists that are range-compatible in order; preserved by absorption, which inserts at the correct position). When only one block remains, its `elements` list is by the invariant sorted, and since fusion/absorption never drop or duplicate elements, it contains exactly the original elements. ■

Termination: Each stumble either produces at least one fusion/absorption (strictly decreasing the block count, which is bounded below by 1) or it produces none. Nothing forces progress on any *given* stumble — this is exactly why the algorithm is not guaranteed to terminate in bounded time without the `max_stumbles` cap (see § 5).

5. Complexity Analysis

5.1 Per-stumble cost

- Random permutation of k blocks: $O(k)$.
- Fuse/absorb scan-and-repeat pass: $O(k)$ per pass, and at most k fusions/absorptions can happen before only 1 block remains, so the inner collapsing loop across the *whole run* does at most $O(n)$ total fusion/absorption work (each element is absorbed/fused into a block exactly once, permanently). This part is cheap and not the bottleneck.

5.2 The real cost: number of stumbles needed

This is where Drunken Merge Sort is bad, and it's worth being precise about *why*.

For a fusion to happen, two blocks whose value ranges are already compatible ($\max(A) \leq \min(B)$) must land **adjacent to each other** in a random permutation. For a random arrangement of k blocks, the probability that two *specific* blocks end up adjacent is:

$$P(\text{adjacent}) = 2 / k \quad (\text{for } k \geq 2)$$

At the very start ($k = n$, all singletons), the number of range-compatible *consecutive-value* pairs is limited — in the best case (already near-sorted values) there are up to $n - 1$ compatible pairs; in the worst case, far fewer are simultaneously useful because fusing "consumes" both blocks. Roughly, per stumble, the expected number of successful fusions is small (bounded by a constant fraction of compatible pairs actually landing adjacent), so:

$$\text{Expected stumbles per single fusion} \approx O(k) \quad \text{where } k = \text{current block count}$$

Summing over the run as k shrinks from n down to 1:

$$E[\text{total stumbles}] \approx \sum (k = n \text{ down to } 1) O(k) = O(n^2) \quad [\text{expected stumbles}]$$

But each stumble itself costs $O(n)$ (permutation + scan), so:

$$\text{Expected total time} \approx O(n^2) \text{ stumbles} \times O(n) \text{ per stumble} = O(n^3) \text{ expected}$$

This is a **rough, favorable-case estimate**. In practice the constant factors and variance are large because early fusions are easy (many singletons, many chances) but the *last* fusion — merging the final two big blocks — requires those two specific large blocks to land adjacent by chance, which does not get easier as blocks get bigger. That final merge dominates runtime for larger n , and its expected wait is $O(n)$ stumbles by itself in the best structured case, but can blow up further if the final two blocks are large and don't have a favorable adjacency count (e.g., only 1 valid adjacency out of k positions).

5.3 Summary table

Case	Behavior
Best case	Input nearly sorted → most blocks fuse in early stumbles → still $\Omega(n)$ stumbles minimum (need at least $n-1$ fusions, each needs a lucky adjacency)
Average case	$\sim O(n^2)$ to $O(n^3)$ -ish expected total work (see caveat above — high variance)
Worst case	Unbounded without <code>max_stumbles</code> ; in practice can dwarf Bogosort's $O(n \times n!)$ for small n because Bogosort only needs the <i>entire</i> permutation to be sorted (a single global event), while Drunken Merge Sort needs a <i>sequence</i> of independent lucky local events
Space	$O(n)$ — blocks partition the original elements, no extra copies beyond bookkeeping
Stable?	Yes — fusion always concatenates in left-to-right order, never reorders equal elements relative to each other
In-place?	No — block/list bookkeeping requires auxiliary structure

Adaptive?	Weakly — nearly-sorted input has more compatible pairs early on, but this doesn't reliably reduce total stumbles because of the "last merge" bottleneck above
-----------	---

Bottom line: Drunken Merge Sort is very likely **worse than Bogosort in practice** for anything but tiny n , because it chains multiple independent low-probability events instead of relying on one. It's kept here as a deliberately-instructive bad algorithm — the value is in the analysis, not the performance.

6. The "sober" version (what this algorithm is secretly imitating)

Strip out the random shuffle and instead do a **single deterministic left-to-right scan**, greedily fusing adjacent sorted runs and merging compatible runs as you go — that's exactly **natural merge sort**:

- Time: $O(n)$ best case (already sorted), $O(n \log n)$ worst case.
- Space: $O(n)$ auxiliary.
- Stable: yes.

Drunken Merge Sort can be thought of as "natural merge sort's fusion logic, sabotaged by re-randomizing block order before every fusion pass instead of scanning once."

7. Practical takeaways for the write-up / README

- Drunken Merge Sort **is correct** (always produces a sorted array if it terminates) — the pseudocode and proof in § 3–4 establish this.
- Drunken Merge Sort **is not efficient** — § 5 shows expected costs that scale much worse than any standard comparison sort, and likely worse than Bogosort for realistic n .
- Its educational value is in (a) implementing block-based fusion logic, (b) reasoning rigorously about expected-time complexity under

randomness, and (c) recognizing its relationship to natural merge sort once the randomness is removed.